

```
format if (numOfdo  
string.replace("czF  
string.replace("czD  
string4replace + te  
plate_type_buffer te  
string4replace + te  
string.replace("czD  
"RT"): for line in  
RIGHTDHRicName = sear  
"RT" + RIGHTDHRicName +  
RIGHTDHRicName = "" op  
buff) import shutil  
().splitlines() for l  
[obj.group(1)] = sea  
ch( c'(\w+)\.
```



For

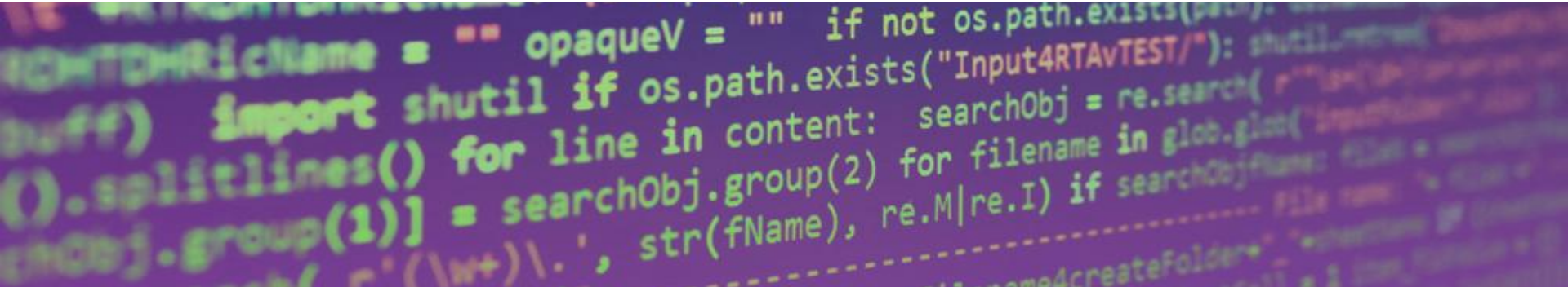
Prof. Luís Guilherme de S. Lopes

Aula 7

- Conceito 'For'
- Como usar em código
- Iteração 'for-each'
- Prática Incluindo HashSet e Switch-case

For

- Ferramenta essencial em diversas linguagens de programação;
- Evita a necessidade de reescrever o código várias vezes, com esse laço é possível realizar uma tarefa repetitiva em um número automático de vezes.



Onde utilizar For ?

Percorrer arrays e listas: Acessar e manipular elementos individualmente.

Repetir ações: Executar um bloco de código ou um número específico de vezes.

Gerar sequências: Criar padrões numéricos ou de caracteres.

Iterar sobre objetos: Acessar propriedades e valores de objetos.

Estrutura

Inicialização: Define o ponto de partida do laço, geralmente atribuindo um valor inicial a uma variável de controle.

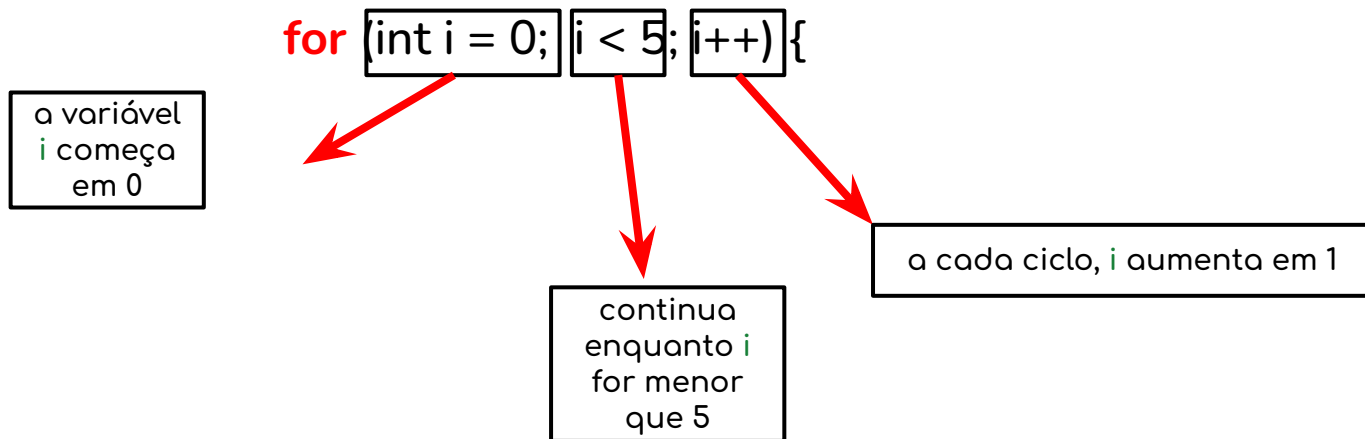
Condição: Determina até quando o laço deve ser executado. Enquanto a condição for verdadeira, o laço continua.

Atualização: Modifica a variável de controle a cada iteração, aproximando-a do ponto em que a condição se tornará falsa.

EXEMPLO

```
for (int i = 0; i < 5; i++) {
```

```
System.out.println("Este é o ciclo número " + (i + 1)); }
```



EXEMPLO

Utilizando o laço "for" para calcular a média de notas de um aluno

```
import java.util.Scanner;
public class CalculadoraMedia {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Quantas notas você deseja inserir? ");
        int quantidadeNotas = input.nextInt();
        double somaNotas = 0;
        for (int i = 1; i <= quantidadeNotas; i++) {
            System.out.print("Digite a nota " + i + ": ");
            double nota = input.nextDouble();
            somaNotas += nota;
        }
        double media = somaNotas / quantidadeNotas;
        System.out.println("A média das notas é: " + media);
        input.close(); // Fecha o Scanner para liberar recursos
    }
}
```

Utilização em Java

Percorrer arrays Acessar e manipular elementos individualmente.

Iterar sobre coleções (List, Set, etc.)

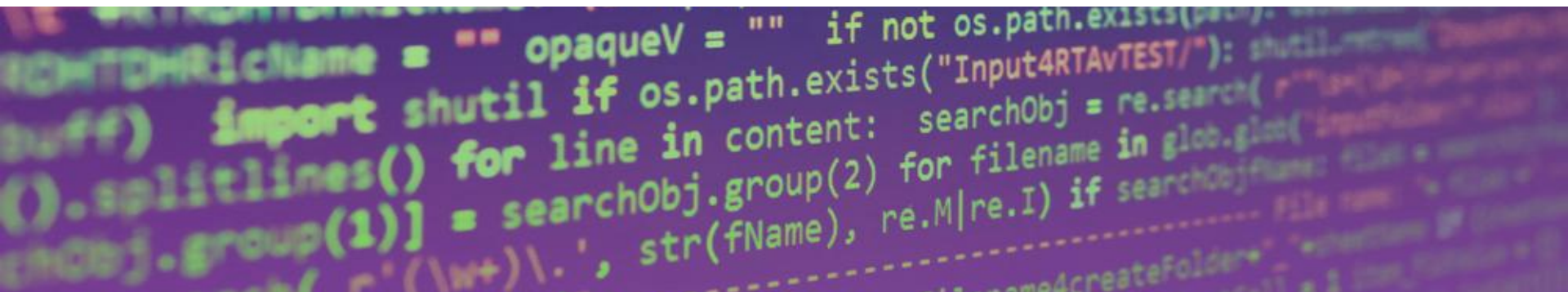
Repetir ações em um número específico de vezes


```
format if (numOfdo  
string.replace("czF  
string.replace("czD  
string4replace + te  
plate_type_buffer te  
string4replace + te  
string.replace("czD  
"RT"): for line in  
RIGHTDHRicName = sear  
"RT"+RIGHTDHRicName+  
RIGHTDHRicName = "" op  
buff) import shutil  
().splitlines() for l  
[obj.group(1)] = sea  
ch("c'(\w+)\\"
```

For-each

For-each

O "For-each" é uma versão simplificada do laço "for" tradicional, projetada especificamente para iterar sobre elementos de arrays e coleções (como listas, conjuntos, etc.). Ele abstrai a necessidade de gerenciar índices explicitamente, tornando o código mais limpo e menos propenso a erros.



Quando utilizar For-each ?

O "for-each" é ideal quando você precisa apenas ler os elementos de uma coleção ou array, sem a necessidade de modificar seus valores ou acessar seus índices.

EXEMPLO

```
for (tipoElemento variavel : colecao) {  
    // Código a ser executado para cada elemento  
}
```

for (tipoElemento variavel : colecao) {

O tipo de dados dos elementos na coleção (por exemplo, int, String, etc.)

Uma variável que receberá cada elemento da coleção a cada iteração.

A coleção ou array sobre a qual você deseja iterar.

EXEMPLO

Utilize “for-each” para contar o número de vogais em uma string fornecida pelo usuário.

```
import java.util.Scanner;

public class ContadorVogais {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Digite uma frase: ");
        String frase = input.nextLine().toLowerCase();
        // Converte para minúsculas

        int contadorVogais = 0;

        for (char letra : frase.toCharArray()) {
            if (letra == 'a' || letra == 'e' || letra == 'i' || letra == 'o' || letra == 'u') {
                contadorVogais++;
            }
        }

        System.out.println("A frase contém " + contadorVogais + " vogais.");

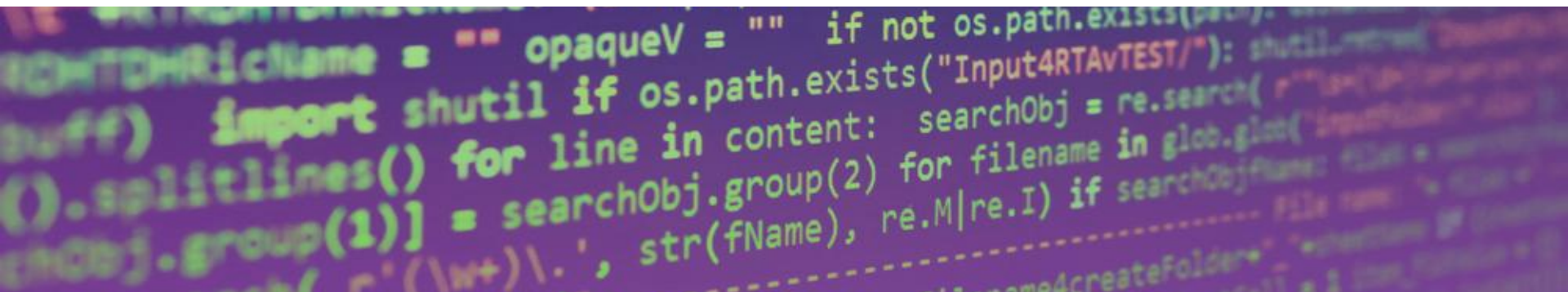
        input.close(); // Fecha o Scanner para liberar recursos
    }
}
```

```
format %s"): (typed
if (numOfdo
string.replace("czF
string.replace("czD
string4replace + te
plate_type_buffer te
string4replace + te
string.replace("czD
"RT"): for line in
TRDHTDHRicName = sear
t" + RTDHTDHRicName +
DHTDHRicName = "" op
buff) import shutil
).splitlines() for l
obj.group(1)] = sea
ch( c'(\w+)\.
```

HashSet

HashSet

- Implementação de um Set, responsável por armazenar valores em uma lista;
- Coleção que não permite elementos duplicados sem garantir uma ordem de inserção.



Quando utilizar HashSet ?

- ➡ Quando você precisa armazenar elementos únicos;
- ➡ Quando a ordem de inserção não é importante;
- ➡ Quando você precisa realizar buscas, inserções e remoções rápidas.

Estrutura

- Essa estrutura permite um acesso rápido aos elementos, pois o código hash indica diretamente onde o objeto está armazenado.

EXEMPLO

```
Set<String> frutas = new HashSet<>();
```

Interface em Java que representa uma coleção de elementos únicos, ou seja, **Set** não permite duplicatas.

Parâmetro do tipo String, onde especifica que os elementos que serão armazenados serão nomes de frutas.

Criando um novo objeto de um construtor da classe HashSet.

EXEMPLO

```
import java.util.HashSet;
import java.util.Set;

public class ExemploHashSet {
    public static void main(String[] args) {
        Set<String> frutas = new HashSet<>();

        frutas.add("Maçã");
        frutas.add("Banana");
        frutas.add("Laranja");
        frutas.add("Maçã"); // Tentativa de adicionar um elemento duplicado (será ignorado)

        System.out.println(frutas); // Saída: [Laranja, Maçã, Banana] (ordem não garantida)

        // Iterando sobre o HashSet
        for (String fruta : frutas) {
            System.out.println(fruta);
        }

        // Verificando se um elemento existe
        if (frutas.contains("Banana")) {
            System.out.println("Banana encontrada!");
        }
    }
}
```

Porque usar HashSet ?

- ➡ Alta eficiência em busca, inserção e remoção de elementos;
- ➡ Elementos únicos, sem exigência de verificação manual para duplicatas.

Métodos Úteis em HashSet

Método	Descrição
<code>add()</code>	Adiciona o elemento especificado ao conjunto.
<code>remove()</code>	Remove o elemento especificado do conjunto.
<code>contains()</code>	Verifica se o conjunto contém o elemento especificado.
<code>size()</code>	Retorna o número de elementos no conjunto.
<code>isEmpty()</code>	Verifica se o conjunto está vazio.
<code>clear()</code>	Remove todos os elementos do conjunto.
<code>iterator()</code>	Retorna um iterador sobre os elementos do conjunto.
<code>toArray()</code>	Converte o conjunto em um array.

Métodos Úteis em HashSet

Método	Descrição
<code>addAll(Collection c)</code>	Adiciona todos os elementos da coleção especificada ao conjunto.
<code>removeAll(Collection c)</code>	Remove todos os elementos do conjunto que estão presentes na coleção especificada.
<code>retainAll(Collection c)</code>	Mantém no conjunto apenas os elementos que também estão presentes na coleção especificada.
<code>hashCode()</code>	Retorna o valor do código hash para este conjunto.
<code>equals(Object o)</code>	Compara este conjunto com o objeto especificado para igualdade.

Métodos Úteis em ArrayList

Método	Descrição
<code>add(elemento)</code>	Adiciona o elemento especificado ao final da lista.
<code>add(elemento)</code>	Insere o elemento especificado em uma posição específica.
<code>remove(elemento)</code>	Remove o elemento na posição especificada.
<code>remove(elemento)</code>	Remove a primeira ocorrência do elemento especificado.
<code>get(elemento)</code>	Retorna o elemento na posição especificada.
<code>set(elemento)</code>	Substitui o elemento na posição especificada.
<code>size()</code>	Retorna o número de elementos na lista.
<code>isEmpty()</code>	Verifica se a lista está vazia.
<code>clear()</code>	Remove todos os elementos da lista.

Métodos Úteis em ArrayList

`indexOf(elemento)`

Retorna o índice da primeira ocorrência do elemento especificado, ou -1 se não for encontrado.

`lastIndexOf(elemento)`

Retorna o índice da última ocorrência do elemento especificado, ou -1 se não for encontrado.

`contains(elemento)`

Verifica se a lista contém o elemento especificado.

`iterator()`

Retorna um iterador sobre os elementos da lista.

`toArray()`

Converte a lista em um array.

`subList(int fromIndex, int toIndex)`

Retorna uma sublista da lista original.

Métodos Úteis em ArrayList

Método	Descrição
<code>addAll(elemento)</code>	Adiciona todos os elementos da coleção especificada ao conjunto.
<code>removeAll(elemento)</code>	Remove todos os elementos do conjunto que estão presentes na coleção especificada.
<code>retainAll(elemento)</code>	Mantém no conjunto apenas os elementos que também estão presentes na coleção especificada.
<code>hashCode()</code>	Retorna o valor do código hash para este conjunto.
<code>equals(Object o)</code>	Comparar este conjunto com o objeto especificado para igualdade.

Exemplos de Uso (HashSet):

```
Set<String> conjunto1 = new HashSet<>(Arrays.asList("a", "b", "c"));  
Set<String> conjunto2 = new HashSet<>(Arrays.asList("c", "d", "e"));
```

```
conjunto1.addAll(conjunto2); // conjunto1 agora contém: [a, b, c, d, e]  
conjunto1.retainAll(conjunto2); // conjunto1 agora contém: [c, d, e]
```

Exemplos de Uso (ArrayList):

```
List<Integer> lista1 = new ArrayList<>(Arrays.asList(5, 1, 3));  
List<Integer> lista2 = new ArrayList<>(Arrays.asList(2, 4));
```

```
lista1.addAll(1, lista2); // lista1 agora contém: [5, 2, 4, 1, 3]  
lista1.sort(Comparator.naturalOrder()); // lista1 agora contém: [1, 2, 3, 4, 5]
```

VAMOS PRATICAR!!!

